

电信前端AI工程师面试

一面（技术基础）

1. 请介绍一下 SSE（Server-Sent Events）的原理，它和 WebSocket 有什么区别？

难度：★★

考点：流式传输、网络协议

答案要点：

SSE 是一种基于 HTTP 协议的服务器向客户端推送数据的技术，它建立一个长连接并以 text/event-stream 格式传输数据。

- 协议层：SSE 基于标准 HTTP，WebSocket 是独立的二进制协议。
- 数据流向：SSE 是单向的（服务器到客户端），WebSocket 是全双工双向通信。
- 断线重连：SSE 内置了自动重连机制，WebSocket 需要开发者手动实现。
- 浏览器支持：SSE 原生支持较好，但在 IE 上需要 polyfill，且有最大连接数限制。

常见坑：

- 忽略了浏览器对同一域名下 SSE 最大连接数（通常为 6 个）的限制。
- 忘记在服务端设置正确的 Content-Type 和 Cache-Control。

追问：

- 如果要在 SSE 中实现双向通信，你会怎么做？
- 为什么 AI 聊天场景通常选择 SSE 而不是 WebSocket？

2. 在前端处理 AI 返回的 Markdown 字符串时，如何实现代码高亮和公式渲染？

难度：★★

考点：DOM 操作、第三方库集成

答案要点：

前端通常使用解析库将 Markdown 转换为 HTML，再配合语法高亮插件和数学公式引擎进行二次渲染。

- 解析库：常用 markdown-it 或 marked 进行基础文档解析。
- 代码高亮：集成 highlight.js 或 prismjs，通过 CSS 选择器对 code 标签进行染色。
- 数学公式：使用 KaTeX 或 MathJax 渲染 LaTeX 格式的公式，需注意异步加载性能。
- 安全防护：必须使用 DOMPurify 等工具过滤 XSS 攻击脚本。

常见坑：

- 在流式输出过程中频繁重新渲染整个 DOM 导致页面闪烁或卡顿。
- 公式渲染库体积过大，影响首屏加载速度。

追问：

- 如何在流式输出（Streaming）过程中实时渲染 Markdown？

3. 说一下 Fetch API 中的 ReadableStream，如何通过它读取流式响应？

难度：★★

考点：异步编程、流处理

答案要点：

Fetch 的 Response 对象暴露了 body 属性，它是一个 ReadableStream，允许我们在数据到达时逐块处理。

- 获取读取器：通过 `response.body.getReader()` 获取 reader 对象。
- 循环读取：使用 `while` 循环调用 `reader.read()`，根据 `done` 状态判断流是否结束。
- 文本解码：使用 `TextDecoder` 将 Uint8Array 字节流转换为可读字符串。
- 错误处理：需捕获网络中断或解析异常，并手动关闭读取器。

常见坑：

- 忘记处理 `read()` 返回的 `value` 可能不完整的情况（如中文字符被截断）。
- 没有正确处理流关闭后的清理逻辑。

追问：

- 如何实现流式读取的超时控制？

4. 手写代码：实现一个简单的防抖函数（Debounce）。

难度：★

考点：闭包、定时器

答案要点：

防抖是指在事件触发后延迟执行，如果延迟时间内再次触发，则重新计时。

- 核心逻辑：利用闭包存储 timer 变量。
- 清除机制：每次触发时先 `clearTimeout`。
- 作用域：确保 `this` 指向正确，并透传参数。

代码块

```
1 function debounce(fn, delay) {
2   let timer = null;
3   return function(...args) {
4     if (timer) clearTimeout(timer);
5     timer = setTimeout(() => {
```

```
6      fn.apply(this, args);
7    }, delay);
8  };
9  }
```

常见坑：

- 忘记返回函数，或者返回的函数中 `this` 丢失。
- 没考虑到立即执行（immediate）的场景需求。

追问：

- 如果要求在开始边界触发（立即执行），代码该如何修改？

5. CSS 如何实现一个打字机效果的动画？

难度：★

考点：CSS 动画、布局

答案要点：

打字机效果可以通过限制容器宽度、隐藏溢出内容并配合 `steps()` 步进函数实现。

- 核心属性：使用 `white-space: nowrap` 防止换行，`overflow: hidden` 隐藏未显示的文字。
- 动画实现：通过 `@keyframes` 改变 `width` 从 0 到 100%。
- 步进函数：`animation-timing-function` 设置为 `steps(字符数)`，模拟逐字跳动。
- 光标效果：利用 `border-right` 模拟闪烁的光标。

常见坑：

- 字符宽度不一致（非等宽字体）导致动画位置偏移。
- 无法动态适配不同长度的文本。

追问：

- 如果文本是动态生成的，CSS 方案还有效吗？

6. 谈谈 Promise.all 和 Promise.allSettled 的区别及应用场景。

难度：★

考点：异步并发控制

答案要点：

这两个方法都用于处理多个并发的 Promise，但对失败的处理逻辑完全不同。

- `Promise.all`：只要有一个失败就立即 reject，适用于强依赖的一组请求。
- `Promise.allSettled`：无论成功失败都会等待所有 Promise 结束，并返回包含状态的对象数组。
- 返回值：`all` 返回结果数组，`allSettled` 返回包含 `status` 和 `value/reason` 的对象数组。

- 场景：all 用于批量获取必要数据，allSettled 用于执行互不影响的独立任务。

常见坑：

- 在 Promise.all 中没有对单个 Promise 做 catch 处理，导致一个失败全盘皆输。

追问：

- 如果来实现一个有并发限制（如最多同时请求 3 个）的 Promise 控制器，思路是什么？

二面（深入原理与项目）

1. 在 AI 问答界面中，当消息列表非常长时，你会如何优化渲染性能？

难度：★★★

考点：长列表优化、虚拟滚动

答案要点：

长列表优化通常采用虚拟滚动（Virtual List）技术，只渲染可视区域内的 DOM 节点。

- 虚拟滚动：计算滚动高度，动态截取数据源中的一部分进行渲染，并用 padding 或 transform 撑起总高度。
- 动态高度：AI 回复长度不固定，需在渲染后通过 ResizeObserver 监听 DOM 高度并更新偏移量索引。
- 节点复用：尽量保持 DOM 结构稳定，减少浏览器的重排与重绘。
- 分片渲染：对于非首屏数据，可以使用 requestIdleCallback 进行后台预处理。

常见坑：

- 滚动过快时出现白屏。
- 动态高度计算不准确导致滚动条抖动。

追问：

- 如果消息中包含图片，图片加载后高度变化如何处理？

2. 如何设计一个 Prompt 模板引擎，支持变量替换和条件判断？

难度：★★

考点：字符串处理、设计模式

答案要点：

Prompt 模板引擎本质上是一个字符串解析器，需要支持占位符替换和简单的逻辑控制。

- 模板语法：定义类似 `{{variable}}` 的占位符，使用正则或 AST 进行匹配。
- 变量注入：提供 context 对象，通过递归或路径查找（如 `user.name`）获取值。
- 逻辑支持：支持 `if/else` 或 `for` 循环，可以通过简单的 DSL（领域特定语言）实现。

- 安全性：对用户输入进行转义，防止 Prompt 注入攻击。

常见坑：

- 嵌套变量处理不当导致解析失败。
- 模板过大时的解析性能问题。

追问：

- 如何在前端实现 Prompt 的版本管理和热更新？

3. 详细说明一下前端如何处理大模型的 Token 流式输出并保证 UI 顺滑？

难度：★★★★

考点：性能优化、用户体验

答案要点：

处理 Token 流的关键在于平衡“实时性”和“渲染频率”，避免高频更新导致主线程阻塞。

- 缓冲区机制：建立一个消息队列（Buffer），将收到的 Token 先存入队列。
- 节流渲染：使用 `requestAnimationFrame` 配合缓冲区，每帧只取出部分 Token 更新到 state。
- 自动滚动：判断用户是否处于页面底部，若是则跟随滚动，若用户手动向上滚动则停止自动跟随。
- 状态管理：使用不可变数据结构更新消息列表，避免不必要的组件全量 re-render。

常见坑：

- 每一个 Token 到达都触发一次 React `setState`，导致低端设备卡死。
- 自动滚动逻辑与用户手动滚动冲突，导致页面“乱跳”。

追问：

- 如何检测用户是否手动向上滚动了？

4. 谈谈你对 RAG（检索增强生成）流程中前端职责的理解。

难度：★★

考点：AI 业务架构

答案要点：

在 RAG 流程中，前端不仅是展示层，还承担了数据预处理、引用溯源展示和反馈收集的任务。

- 引用标注：解析模型返回的引用索引，在 UI 上实现点击跳转到原文片段或高亮显示。
- 文档上传与切片：前端可预先对文档进行格式校验、简单的文本清洗或预览。
- 检索反馈：提供“赞/踩”功能，并将用户反馈与特定的检索片段关联，用于优化 Embedding 效果。
- 状态可视化：展示 RAG 的中间步骤（如：正在检索知识库、正在总结内容）。

常见坑：

- 引用链接失效或指向错误的文档片段。

- 忽略了多模态（如 PDF 中的表格、图片）的展示需求。

追问：

- 如果检索出的参考文档非常多，前端如何优雅地展示？

5. 如何在前端实现一个 AI Agent 的任务拆分与执行过程的可视化？

难度：★★★

考点：复杂状态机、UI 设计

答案要点：

可视化 Agent 过程需要将复杂的后台逻辑抽象为状态流或树状图。

- 状态追踪：定义 Agent 的生命周期（思考中、调用工具中、执行中、已完成）。
- 步骤展示：使用步骤条（Steps）或时间轴（Timeline）展示任务拆分后的 Sub-tasks。
- 工具调用详情：展示 Agent 调用的 API 名称、入参及返回结果，通常使用折叠面板。
- 交互干预：在关键节点（如高危操作）提供人工确认（Human-in-the-loop）的交互界面。

常见坑：

- 步骤过多时 UI 过于拥挤。
- 异步状态同步不及时导致前端显示与后端执行脱节。

追问：

- 如果 Agent 陷入死循环，前端如何感知并提醒用户？

6. 谈谈 Web Worker 在 AI 前端应用中的使用场景。

难度：★★

考点：多线程编程

答案要点：

Web Worker 适合处理计算密集型任务，防止阻塞主线程（UI 线程）。

- 文本处理：大规模文本的清洗、分词或 Embedding 计算（如果是前端离线模型）。
- 语法高亮：对超长代码块进行高亮解析。
- 离线模型运行：使用 Transformers.js 等库在浏览器端运行小模型。
- 数据转换：将复杂的 JSON 响应转换为可视化图表所需的格式。

常见坑：

- Worker 与主线程通信（postMessage）的数据拷贝开销过大。
- 无法在 Worker 中直接操作 DOM。

追问：

- 如何优化 Worker 间的大数据传输？（提示：Transferable Objects）

7. 你们是如何处理 AI 响应过程中的网络中断或异常的？

难度：★★

考点：容错处理、异常监控

答案要点：

异常处理需要覆盖网络层、协议层和业务逻辑层。

- 重试机制：针对 5xx 错误或网络超时，实现指数退避（Exponential Backoff）的自动重试。
- 状态恢复：记录已收到的部分 Token，重连后支持从断点继续展示或明确提示中断。
- 交互提示：提供清晰的错误信息（如“模型拥挤”、“内容违规”）并给出重试按钮。
- 监控告警：通过 Sentry 等工具捕获流式传输中的解析异常，并记录上下文。

常见坑：

- 发生错误后，Loading 状态没有清除，导致界面永久挂起。
- 没处理后端返回的 Partial Content 导致的 JSON 解析失败。

追问：

- 如何区分是网络抖动还是后端服务崩溃？

8. React 中，如何避免 AI 聊天组件因为父组件更新而产生不必要的重渲染？

难度：★★

考点：React 性能优化

答案要点：

AI 聊天界面状态频繁变化，必须严格控制组件更新范围。

- 记忆化：使用 `React.memo` 包裹消息项组件，通过自定义比较函数判断 props 是否变化。
- 状态下放：将输入框的 state 留在输入框组件内部，不要提升到顶层。
- 钩子优化：使用 `useMemo` 缓存复杂的 Markdown 解析结果，使用 `useCallback` 缓存事件处理函数。
- 引用稳定：确保传递给子组件的数组或对象引用在数据未变时保持一致。

常见坑：

- 在 render 函数中直接定义匿名函数或对象。
- 滥用 Context 导致整个消息列表全量更新。

追问：

- 如果使用 Zustand 或 Redux，如何实现局部状态订阅？

三面（架构与综合能力）

1. 如果让你从零设计一个企业级 AI 助手的前端架构，你会考虑哪些核心模块？

难度：★★★

考点：系统架构设计

答案要点：

一个成熟的 AI 前端架构应具备高扩展性、高性能和良好的用户体验。

- 通信层：封装 SSE/WebSocket 适配器，支持流式协议解析、心跳检测和自动重连。
- 渲染引擎：集成 Markdown、Latex 解析，支持多模态（图片、文件、图表）渲染插件。
- 状态管理：管理对话上下文（Context）、会话历史、Agent 执行状态。
- 安全与合规：实现敏感词过滤、输入长度限制、Prompt 注入防护。
- 监控与评估：埋点收集用户反馈、统计首 Token 延迟（TTFT）和生成速度。

常见坑：

- 模块耦合太紧，导致更换大模型或通信协议时需要重构。
- 忽略了离线存储和多端同步的需求。

追问：

- 如何实现“插件化”的工具调用展示？

2. 在电信业务场景下，AI 前端如何处理数据安全和隐私保护？

难度：★★★

考点：数据安全、合规性

答案要点：

电信数据敏感度高，前端需配合后端建立多重防护体系。

- 脱敏处理：在发送 Prompt 前，通过正则或本地模型识别并拦截手机号、身份证等敏感信息。
- 传输加密：全量使用 HTTPS，对敏感 Payload 进行二次加密。
- 权限控制：严格校验会话 Token，防止越权访问历史对话。
- 审计日志：记录用户的所有输入输出行为，用于事后溯源，但前端不存储敏感明文。

常见坑：

- 将敏感信息缓存在 localStorage 中被恶意脚本读取。
- 在报错日志中泄露了用户的输入内容。

追问：

- 如果需要在前端实现本地脱敏，你会选择什么技术方案？

3. 谈谈你对“前端工程化”在 AI 领域新挑战的看法。

难度：★★★

考点：行业视野、工程化

答案要点：

AI 改变了前端的开发模式和交付内容，带来了新的工程化需求。

- Prompt 管理：Prompt 成为了一种新的“代码”，需要版本控制、灰度发布和单元测试。

- 评测自动化：前端需要参与构建模型输出质量的评测工具（如对比不同模型的渲染效果）。
- 资源管理：大模型相关的 Web 资源（如 WASM 权重文件）体积巨大，需优化 CDN 和缓存策略。
- 交互范式变革：从 GUI（图形界面）向 LUI（语言界面）转型，前端需探索更自然的交互方式。

常见坑：

- 依然沿用传统的 Web 开发思维，忽视了 AI 输出的不确定性。

追问：

- 你认为 AI 会取代前端开发吗？为什么？

4. 聊聊你在过去一年中遇到的最难的 AI 相关前端技术问题，你是如何解决的？

难度：★★★

考点：问题解决能力、项目深度

答案要点：

此题没有标准答案，重点考察候选人的思考深度和技术攻关能力。

- 问题描述：清晰定义问题的背景、技术瓶颈和影响范围（如：流式渲染导致的严重掉帧）。
- 排查过程：使用过哪些工具（Chrome DevTools, Performance tab）和方法进行定位。
- 解决方案：对比过哪些方案，最终为什么选择了这一种（如：引入了自定义的缓冲区调度算法）。
- 最终效果：通过量化指标（如：FPS 从 20 提升到 60）证明解决效果。

常见坑：

- 描述过于笼统，没有体现出技术细节。
- 解决方案听起来像是随机尝试的结果，缺乏逻辑性。

追问：

- 如果现在重新做一次，你会尝试什么不同的方法？

5. 如何评估一个 AI 前端应用的用户体验？有哪些关键指标？

难度：★★

考点：产品思维、性能指标

答案要点：

AI 应用的体验评估需要结合传统 Web 指标和 AI 特有指标。

- TTFT（Time to First Token）：首个字符返回的时间，直接影响用户的感知速度。
- 生成速度：平均每秒生成的 Token 数，反映了系统的吞吐能力。
- 交互成功率：用户输入到获得有效回复的比例，需剔除网络错误和模型幻觉。
- 用户满意度（CSAT）：通过点赞/点踩等显性反馈收集。
- 纠错成本：用户手动修改 Prompt 或重新生成的频率。

常见坑：

- 只关注 LCP/FID 等传统指标，忽略了 AI 响应的特殊性。

追问：

- 如果 TTFT 过长，前端可以做哪些交互补偿？

6. 针对低端设备，你会如何优化运行在浏览器端的 AI 模型（如 Transformers.js）？

难度：★★★

考点：WebGPU、性能极限优化

答案要点：

浏览器端运行模型对计算资源要求极高，需从模型压缩和硬件加速两方面入手。

- 模型量化：使用 4-bit 或 8-bit 量化后的模型，显著降低显存占用和计算量。
- 硬件加速：优先使用 WebGPU 接口，相比 WebGL 能提供更高效率的并行计算能力。
- 缓存策略：利用 Cache API 存储模型权重文件，避免重复下载。
- 算力分级：根据设备性能检测，动态决定是在本地运行小模型还是调用云端大模型。

常见坑：

- 没做设备检测直接加载大模型导致浏览器崩溃。
- 忽略了 WebGPU 的兼容性问题。

追问：

- WebGPU 相比 WebGL 在 AI 计算上的核心优势是什么？